

Creating and Maintaining a Build Log

What a build log is

Your build log is a running record you keep *as you build*, over multiple sessions. It is not a summary you write at the end. It has two parts:

- A short, living list of the **decisions you have locked** (controls, core systems, scope, constraints), which you and your AI keep current and re-read to stay consistent.
- An **entry per session** capturing what you did, decided, changed and learned, and where things stand for next time.

Why it is worth keeping, and the first reason is the real one:

1. **It makes your build better and easier.** Across a multi-session build this is working memory. Your AI can re-read your locked decisions so it stops contradicting choices you already made, and you can pick up cleanly between sessions instead of re-deriving where you were. Creators who keep one find it pays for itself quickly.
2. **It is a required submission.** This is an AI-native competition, so prototypes are built by prompting AI tools. Your build log is how you show your prototype was built that way, and it gives judges insight into your process. It is **not scored on quality**, and it is kept confidential (see File format and privacy below).

Required: every entry must include a build log. That is the only hard requirement on this page. How you produce it is up to you.

One easy way: have your AI keep it for you

The least effort is not to write it by hand at all. If you want to go this route, paste something like this into your AI tool when you start building, and it will maintain the log as you go:

None

I'm building a game prototype for a competition over the next few weeks. As we work, create and continuously maintain a single build log file (buildlog.md) that is genuinely useful for both of us to refer back to across sessions. Keep two parts:

PART 1 - "Decisions locked so far": a running list at the top of the file of the design and technical decisions we've settled

(controls, core systems, scope, constraints, art approach). Keep it current as decisions change, and re-read it at the start of each session so you stay consistent with what we already decided.

PART 2 - an entry per work session (dated or numbered), covering:

- Which AI tool(s) we used
- What we built or changed this session
- The key decisions I made, and why
- Anything I pivoted on, and what changed my thinking
- What I changed after playtesting: what felt off, and what we tuned
- The biggest problems we hit, and how we solved them
- What I learned this session
- Where things stand, and what's next (so I can pick up cleanly next time)

Update the log as we go, not at the end. Capture enough detail to be genuinely useful to look back on, not just a summary. Be honest, including what didn't work. Re-read the log when I ask, to stay consistent. This log is both my working reference and a required competition submission.

Then keep building normally. When you are done, export the log as a `.md` file and submit it.

Using a chat tool that cannot keep a file? Some AI tools cannot maintain a file or remember it across sessions. That is fine. Ask it to produce or update the full log at the end of each session, save that text yourself in a `.md` file, and paste it back at the start of your next session so the tool stays consistent with your earlier decisions. Same log, you just hold the file.

What the log needs to cover

If your AI maintains it for you, this is already handled. If you are writing it yourself, make sure the log has:

Part 1, "Decisions locked so far" — a short running list at the top: the design and technical decisions you have settled (controls, core systems, scope, constraints, art). Update it when a decision changes, and note the change in that session's entry.

Part 2, an entry per session (dated or numbered): what you built, key decisions and why, pivots, what changed after playtesting, the biggest problems and how you solved them, what you learned, and where things stand for next time.

Details are welcome. Usefulness matters more than length, and it is not scored, so be honest, including what did not work.

Did not set one up at the start?

No problem, and you do not need to reconstruct it from memory. The easiest fix is to ask the AI you built with to generate the log from its own build history:

None

```
We just built a game prototype together over several sessions.  
Based on our full conversation and build history, generate a build  
log now as buildlog.md. Include:
```

- "Decisions locked so far": the design and technical decisions we settled (controls, core systems, scope, constraints, art approach).
- An entry per work session (dated or numbered) covering: what we built, the key decisions and why, any pivots, what changed after playtesting, the biggest problems and how we solved them, and what we learned.

```
Be honest, including what didn't work. Make it genuinely useful,  
not just a summary.
```

File format and privacy

- **File format:** Markdown (`.md`), named `buildlog.md`.
- Do **not** include personal or sensitive information, or other people's names.
- Your build log is shared **only with internal teams and judges**. It is **never published publicly**, and it is **not used to train AI models**.

Example

None

```
# Build Log: Reef Keeper  
Genre: Simulation & Management
```

Decisions locked so far

- Core loop: earn credits from a healthy tank, spend to add creatures, keep the ecosystem stable.
- Genre floor: a resource the player invests in, a system response they can read, and visible growth across the session. All three present.
- Water chemistry = 3 values only (oxygen, pH, waste). Readability over realism.
- Controls: tap-to-place, tap-to-remove. Portrait, mobile-first.
- Scope: 6 creature types, 1 tank, win = stable 60s, lose = tank crash.
- Difficulty comes from higher-value species being more sensitive, not from artificial limits.

Session 1 (day 1): core loop

Tool(s): [AI tool]

What I built: an empty tank, a credits counter, and placing 6 creature types that each cost credits.

Key decisions: kept chemistry to 3 values so it stays legible; tap-to-place for simple mobile controls.

Pivots: the AI first proposed a 10-variable ecosystem. Cut it to 3. A player can't feel 10 interacting numbers and I couldn't balance it.

What changed after playtesting: placing felt flat, so I added live chemistry meters that react instantly. That made each placement a real decision.

Biggest problem: with 10 variables the tank crashed randomly with no legible cause. Cutting to 3 fixed it.

What I learned: for a management game, fewer legible systems beat many hidden ones. Readability is the fun.

Where things stand / next: core placement loop works and feels good.

Next: add progression and a win/lose state.

Session 2: progression and win/lose

Tool(s): [AI tool]

What I built: higher-value species that unlock as tank value rises, a win state (stable 60s), a lose state (crash).

Key decisions: higher-value species are more sensitive, so difficulty rises naturally within a single session.

What changed after playtesting: the crash came with no warning and felt unfair, so stressed fish now visibly hide as an early signal.

Biggest problem: the AI said the win state worked, but it never triggered. I had to play it, find the bug, and point it out before it fixed the timer.

What I learned: never trust "done" without playing it.

Where things stand / next: full loop is playable start to finish.

Next: a balance and feel pass.

Session 3: feel and balance pass

Tool(s): [AI tool]

What I built: no new systems, just tuning (credit income, species costs, drift speed).

Pivots: planned 12 species, but it was more fun with fewer distinct ones, so I dropped to 6 with clearer roles. Depth over breadth. (Updated the locked list.)

What changed after playtesting: income was too high, so spending had no tension. Halved starting credits and the "what next" decision got interesting.

Biggest problem: pooled fish kept their old color when reused. Had to reset appearance on reuse.

What I learned: for this genre the fun is the economy tension, not the number of things. The balance pass mattered more than any feature.

Where things stand / next: prototype is submission-ready.

Next: final playtest with a friend, then package.